

will separate a file into multiple records based on the occurrence of \$ in the text.

```
[testuser@localhost ~]$ cat testfile
abc$def$ghi$kim
[testuser@localhost ~]$ awk -v RS="$" '{print}' testfile
abc
def
ghi
kim
```

Output is printed based on the value of the Output Record Separator (ORS) variable. The default value of ORS is the newline character (\n). We will get into the details of this small program shortly.

Fields are, by far, the most important feature of Awk. Each input record is further divided into fields based on a Field Separator (FS) variable. Like RS, FS can be any single character or a regular expression. By default, it's a space character. If it's a space, records are separated into fields if any white space character (space, tab or newline) occurs in the text. Fields are what make Awk so useful for text manipulation. Most of the time, one has to search for a particular text at a particular location in the line. That's where fields come in handy.

Awk assigns the value of the field to a built-in variable \$n, based on the order of occurrence of that field in the record. For example, for a line containing the text "Hello, World!", if the FS value is ", \$1 becomes "Hello" and \$2 becomes "World.". These variables are valid only till the current input record. There's a special variable \$0 that is equal to the whole input record. Fields may be referenced through constants like \$1, \$2, etc, or through variables. For example if N=5, \$N may be used instead of \$5.



Tip: The FS value can also be assigned using a command-line switch `-F`.

In short, each input file is divided into records based on the RS variable, and each record is further divided into fields based on the FS variable. Field values are assigned to special variables based on their occurrence, with the first field being \$1, the second \$2 and so on. The special variable \$0 is equivalent to the input record. Now, we will see how an Awk program works.

Anatomy of an Awk program

An Awk program mainly consists of patterns and actions. It can also include variable assignments and function definitions, but the most important parts are patterns and the actions that are to be taken when those patterns occur in the input text. Each pattern specified is checked against each input line read, and the actions defined for that pattern are executed. Either the pattern may be missing, in which case the defined action is executed for all input lines, or the action may

be missing—in which case the default action of printing the current input line is executed, i.e., `{print $0}`.

The syntax of an Awk program is `'PATTERN{ACTION}'`. Action statements are enclosed within `{ }` and the whole program is enclosed within single quotes when executed directly on the command line. An Awk program can also be saved in a file and executed using the `-f` switch. In such a case there's no need to enclose the program within quotes.

```
[testuser@localhost ~]$ cat testfile
abc.def.ghi.klm
abc.test.ghi.dsfdss
abc.def.test.kshsf
```

```
[testuser@localhost ~]$ cat myfile
/test/ { print } # pattern{ action }. Print those lines
which contain the pattern "test"
```

```
[testuser@localhost ~]$ awk -F"." -f myfile testfile
abc.test.ghi.dsfdss
abc.def.test.kshsf
```

Pattern forms

Patterns can be specified in various forms like:

- **Regular expressions:** A pattern can be any regular expression. Gawk supports extended regular expressions. Thus patterns containing character classes like `[alpha:]`, `[digit:]`, `[lower:]`, etc, are also supported. A detailed discussion of the regular expressions is beyond the scope of this article.
- **Relational expressions:** Relational expressions utilising the operators `&&`, `||`, `!` can be used to match complex patterns. The C ternary operator `?` is also supported for pattern matching. In this case, an expression is specified as `pattern1?pattern2:pattern3`. If `pattern1` is true, `pattern2` is evaluated, else, `pattern3` is evaluated.
- **Pattern 1, Pattern 2:** The man page states that this form specifies a range of text wherein the actions specified are executed for all the record lines starting with a record matching `pattern1`, continuing until a record matching `pattern2`. (See the examples at the end.)
- **BEGIN and END patterns:** There are two special patterns defined in Awk—**BEGIN** and **END**. Actions specified for **BEGIN** are executed at the start of the program before any input records are read. Thus, it's a good place for any global variable initialisation or to perform any tasks that should precede the start of the input. Similarly, actions specified for the **END** pattern are executed after all the input records have been read and the actions specified for other patterns have been executed. Actions for **BEGIN** and **END** patterns are executed only once, and are independent of the number of input records.